# SOC 2 Logging Architecture (Supabase + FastAPI)

## 1) Objective

Design a SOC 2 aligned logging architecture with:

- `global_admin`: can view audit logs for everyone.
- `admin`: can view logs for self + users in their tenant/scope.
- `user`: can view only own logs.
- all critical actions written to Supabase as immutable audit events.

This document is implementation-ready and does not require code changes now.

---

## 2) High-Level Architecture

### A. Two Logging Streams

1. **Operational Logs (App Logs)**
   - Runtime diagnostics for engineering.
   - JSON logs from FastAPI/Uvicorn.
   - Includes `request_id`, `route`, `duration_ms`, `status_code`, exception details (redacted).
   - Forward to SIEM/log platform.

2. **Audit Logs (Compliance Logs)**
   - Evidence logs for SOC 2.
   - Written to Supabase `public.audit_events`.
   - Immutable append-only, queryable with role-based RLS.
   - Covers security-sensitive and business-critical actions.

### B. Write Flow

1. Request enters FastAPI with correlation ID (`request_id`).
2. Auth context resolved (user id, role, tenant id).
3. Endpoint executes privileged action.
4. Audit event emitted to Supabase with `action_id`, actor, target, result, metadata.
5. Event visible by role per RLS policies.

---

## 3) Role Visibility Model

## Required behavior

- `global_admin` -> read all audit events.
- `admin` -> read own events + users/viewers/guests in same tenant.
- `user` -> read own events only.

## Recommended data model assumptions

- `app_users(user_id, tenant_id, ...)`
- `user_roles(user_id, role)`
- `has_role(auth.uid(), 'admin')` already supports `global_admin`.

---

## 4) Supabase Schema (Recommended)

Use this as your canonical schema for audit logging.

*--- code block ---*

```
-- 1) Main immutable audit events table
create table if not exists public.audit_events (
  id uuid primary key default gen_random_uuid(),

  -- Correlation
  request_id text not null,
  trace_id text,
  session_id text,

  -- Actor
  actor_user_id uuid references auth.users(id),
  actor_role public.app_role not null,
  actor_tenant_id uuid references public.tenants(id),

  -- Action identity
  action_id text not null,          -- machine ID e.g. iam.role.assign
  action_label text not null,       -- human label e.g. "Assign role"
  category text not null,           -- iam, auth, device, model, security

  -- Target
  resource_type text not null,      -- user, device, model, role, policy, etc.
  resource_id text,
  target_user_id uuid references auth.users(id),
  target_tenant_id uuid references public.tenants(id),

  -- Outcome
  outcome text not null check (outcome in ('success','failure','denied')),
  status_code int,
  reason text,

  -- Environment
  ip_address inet,
  user_agent text,
  source text not null default 'backend_api', -- backend_api, scheduler, system

  -- Change evidence (redacted/minimal)
  before_state jsonb,
  after_state jsonb,
  diff jsonb,
  metadata jsonb not null default '{}'::jsonb,

  -- Compliance controls
  contains_sensitive boolean not null default false,
  redaction_version text,
  created_at timestamptz not null default now()
);

-- 2) Optional action dictionary for governance
create table if not exists public.audit_action_catalog (
  action_id text primary key,
  category text not null,
  severity text not null check (severity in ('low','medium','high','critical')),
  description text not null,
  requires_ticket boolean not null default false,
  requires_mfa boolean not null default false,
  enabled boolean not null default true,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

-- 3) Indexes for investigations and dashboards
```

```
create index if not exists idx_audit_events_created_at on public.audit_events (created_at desc);
create index if not exists idx_audit_events_action_id on public.audit_events (action_id);
create index if not exists idx_audit_events_actor_user on public.audit_events (actor_user_id, created_at
  desc);
create index if not exists idx_audit_events_target_user on public.audit_events (target_user_id, created_at
  desc);
create index if not exists idx_audit_events_tenant on public.audit_events (actor_tenant_id, created_at desc);
create index if not exists idx_audit_events_outcome on public.audit_events (outcome, created_at desc);
```
*--- code block ---*

---

## 5) Immutability Controls (SOC 2 Critical)

*--- code block ---*
```
-- No updates/deletes to audit rows except service role / owner
create or replace function public.block_audit_event_mutation()
returns trigger
language plpgsql
security definer
as $$
begin
  raise exception 'audit_events are immutable';
end;
$$;

drop trigger if exists trg_block_audit_events_update on public.audit_events;
create trigger trg_block_audit_events_update
before update on public.audit_events
for each row execute function public.block_audit_event_mutation();

drop trigger if exists trg_block_audit_events_delete on public.audit_events;
create trigger trg_block_audit_events_delete
before delete on public.audit_events
for each row execute function public.block_audit_event_mutation();
```
*--- code block ---*

---

## 6) RLS Policies for Role-Based Access

*--- code block ---*
```
alter table public.audit_events enable row level security;

-- Global admin: see all
drop policy if exists "global admin read all audit events" on public.audit_events;
create policy "global admin read all audit events"
on public.audit_events
for select
using (public.has_role(auth.uid(), 'global_admin'));

-- Admin: same tenant + own + non-global-admin actors/targets in same tenant
drop policy if exists "admin read tenant audit events" on public.audit_events;
create policy "admin read tenant audit events"
on public.audit_events
for select
using (
  public.has_role(auth.uid(), 'admin')
  and exists (
    select 1
    from public.app_users me
    where me.user_id = auth.uid()
      and (
```

```
      audit_events.actor_tenant_id = me.tenant_id
      or audit_events.target_tenant_id = me.tenant_id
    )
  )
);

-- User: own actor events OR events where user is target
drop policy if exists "user read own audit events" on public.audit_events;
create policy "user read own audit events"
on public.audit_events
for select
using (
  auth.uid() = actor_user_id
  or auth.uid() = target_user_id
);

-- Write: only authenticated backend context.
-- If backend writes with service role key, RLS is bypassed.
drop policy if exists "authenticated insert audit events" on public.audit_events;
create policy "authenticated insert audit events"
on public.audit_events
for insert
with check (auth.uid() is not null);
```
*--- code block ---*

Note:
- If you exclusively write with service role from backend, insert policy is mostly informational.
- Keep writes backend-only for privileged operations; do not let frontend perform direct privileged writes.

---

## 7) Action ID Naming Standard

Format:

*--- code block ---*

Examples:

- `iam.role.assign`
- `iam.user.create`
- `device.link.confirm`
- `security.auth.login_failed`
- `pod.request.approve`

Rules:

- Lowercase, dot-separated, stable over time.
- Never rename old IDs (create new ID if semantics change).
- `action_label` can be user-friendly and editable; `action_id` must be immutable.

---

## 8) Canonical Action Catalog (Recommended)

## IAM / RBAC

- `iam.user.create`
- `iam.user.password_reset`
- `iam.user.disable`
- `iam.user.enable`
- `iam.role.assign`

- `iam.role.revoke`
- `iam.role.escalation_attempt_denied`
- `iam.permission.override`

### Authentication / Session

- `security.auth.login_success`
- `security.auth.login_failed`
- `security.auth.logout`
- `security.auth.token_refresh`
- `security.auth.mfa_challenge`
- `security.auth.mfa_failed`

### Admin / Global Admin Governance

- `governance.admin.action`
- `governance.global_admin.action`
- `governance.break_glass.start`
- `governance.break_glass.end`
- `governance.policy.change`

### Device and Pairing

- `device.register`
- `device.checkin`
- `device.deactivate`
- `device.reactivate`
- `device.pairing.start`
- `device.pairing.confirm`
- `device.pairing.expired`

### Pod and Model Operations

- `pod.request.create`
- `pod.request.approve`
- `pod.request.reject`
- `model.allocate`
- `model.deallocate`

### Data and Security

- `security.data.export`
- `security.data.delete_request`
- `security.rls.policy_violation`
- `security.rate_limit.triggered`
- `security.api.access_denied`

---

## 9) Demo Event Payloads

### Example A: Global admin assigns admin role

*--- code block ---*

```
{
  "request_id": "req_20260313_8f2a",
  "actor_user_id": "4d3f...a1",
  "actor_role": "global_admin",
  "actor_tenant_id": "aa10...44",
  "action_id": "iam.role.assign",
  "action_label": "Assign role",
```

```
  "category": "iam",
  "resource_type": "user_roles",
  "resource_id": "target_user_uuid",
  "target_user_id": "8bc1...f0",
  "target_tenant_id": "aa10...44",
  "outcome": "success",
  "status_code": 200,
  "source": "backend_api",
  "before_state": {"role": "user"},
  "after_state": {"role": "admin"},
  "diff": {"role": ["user", "admin"]},
  "metadata": {"ticket_id": "SEC-1492", "reason": "Team lead promotion"}
}
```
*--- code block ---*

### Example B: Admin approves pod request

*--- code block ---*
```
{
  "request_id": "req_20260313_b20d",
  "actor_user_id": "2e8b...9d",
  "actor_role": "admin",
  "actor_tenant_id": "aa10...44",
  "action_id": "pod.request.approve",
  "action_label": "Approve pod activation request",
  "category": "pod",
  "resource_type": "pod_activation_requests",
  "resource_id": "request_uuid",
  "target_user_id": "6f2c...19",
  "target_tenant_id": "aa10...44",
  "outcome": "success",
  "status_code": 200,
  "metadata": {"model_id": "llama-insurance-v2", "domain": "insurance"}
}
```
*--- code block ---*

### Example C: User reads own event history

*--- code block ---*
```
select *
from public.audit_events
where actor_user_id = auth.uid()
   or target_user_id = auth.uid()
order by created_at desc
limit 100;
```
*--- code block ---*

---

## 10) Query Examples by Role

### Global admin: all critical failures in 24h

*--- code block ---*
```
select created_at, action_id, actor_user_id, resource_type, resource_id, outcome, reason
from public.audit_events
where outcome in ('failure', 'denied')
  and created_at >= now() - interval '24 hours'
order by created_at desc;
```
*--- code block ---*

### Admin: role changes in own tenant last 7 days

--- *code block* ---
```
select created_at, actor_user_id, target_user_id, before_state, after_state, metadata
from public.audit_events
where action_id in ('iam.role.assign', 'iam.role.revoke')
  and created_at >= now() - interval '7 days'
order by created_at desc;
```
--- *code block* ---

### User: own security events

--- *code block* ---
```
select created_at, action_id, outcome, ip_address, user_agent
from public.audit_events
where (actor_user_id = auth.uid() or target_user_id = auth.uid())
  and action_id like 'security.auth.%'
order by created_at desc;
```
--- *code block* ---

---

## 11) Integration Pattern with FastAPI + Supabase

Use one centralized audit writer service from backend routes.

Pseudo-contract:

--- *code block* ---
```
await audit.log_event(
    request_id=req_id,
    actor_user_id=user.id,
    actor_role=user.role,
    actor_tenant_id=user.tenant_id,
    action_id="iam.role.assign",
    action_label="Assign role",
    category="iam",
    resource_type="user_roles",
    resource_id=target_user_id,
    target_user_id=target_user_id,
    target_tenant_id=target_tenant_id,
    outcome="success",
    status_code=200,
    ip_address=client_ip,
    user_agent=user_agent,
    before_state={"role": old_role},
    after_state={"role": new_role},
    metadata={"ticket_id": ticket_id}
)
```
--- *code block* ---

Write path:

- Backend -> Supabase REST (`/rest/v1/audit_events`) with service role key.
- Keep this insert call non-blocking where practical (queue/buffer if high volume).

---

## 12) SOC 2 Control Mapping (Quick)

- **CC6/CC7 (Access + Monitoring):**
  - role-based access checks + failed/denied logs.
- **CC7.2 (Anomaly detection):**
  - alert on `iam.role.assign`, `governance.break_glass.*`, repeated `security.auth.login_failed`.

- **CC8 (Change management):**
  - policy/config change action IDs with ticket references.
- **Evidence readiness:**
  - immutable logs, retention policy, and monthly export reports.

---

## 13) Retention and Alerting Recommendations

- Hot searchable retention: `90 days`.
- Archive retention: `365+ days` (based on policy/legal needs).
- Alerts (real-time):
  - any `iam.role.assign` to `global_admin`
  - any `governance.break_glass.start`
  - 5+ failed admin actions in 10 minutes
  - repeated denied access to protected APIs

---

## 14) Final Notes for Your Current Setup

- Your existing `audit_logs` table can be migrated to `audit_events` or extended in-place.
- Keep privileged writes backend-only to preserve trust and evidence integrity.
- Ensure frontend does not directly perform privileged role mutations when backend is unavailable.

This architecture gives you a clean SOC 2 story: complete traceability, strong access boundaries, and audit evidence quality for `global_admin`, `admin`, and `user`.